

SparseTreePIR: Active-Interval Coloring for Batch Private Information Retrieval of Sparse Merkle Proofs

Anonymous Author(s)

Abstract—Sparse Merkle Trees (SMTs) provide compact membership proofs over large logical address spaces by combining non-default sibling digests with public default hashes. A standard proof request, however, reveals the queried coordinate and the corresponding authentication path. This paper studies target-private retrieval of SMT membership proofs for a fixed public snapshot. The key observation is that the proof verified by the client and the database queried through private information retrieval (PIR) should not be the same object: the verifier still needs a standard height- h SMT proof, while the PIR backend only needs to retrieve proof entries that cannot be reconstructed from public default hashes.

We present SparseTreePIR, a PIR-based retrieval layout for this setting. SparseTreePIR stores only non-default sibling digests that can appear in membership proofs of occupied leaves. For each such digest, setup records the interval of target leaves whose proofs require it. These intervals form a laminar structure, enabling a coloring in which any membership proof needs at most one real record from each color subdatabase. During online retrieval, the client sends one PIR query to every color subdatabase, using dummy queries when a color is not needed, and then reconstructs the ordinary height- h SMT proof locally with the public default-hash chain.

We further introduce ActiveBalance, a load-balancing algorithm that reduces the largest color subdatabase while preserving the same one-query-per-color retrieval pattern. On six SMT workloads, the resulting retrieval width is only 9–17 for height-128 and height-256 proofs. Compared with height-preserving pruned TreePIR, SparseTreePIR uses $9.5\times$ fewer subqueries and has a $2.5\times$ smaller largest searched subdatabase on height-128 workloads. Implementation experiments with SimplePIR and PIANO show $2.1\times$ – $2.3\times$ lower online communication than an optimistic PBC-style SMT baseline, without modifying the underlying PIR primitive.

Index Terms—Sparse Merkle tree, private information retrieval, batch PIR, authenticated map, membership proof, proof retrieval.

1. Introduction

Sparse Merkle Trees (SMTs) are Merkle trees over large logical address spaces. A height- h SMT has 2^h possible leaf coordinates, but only a small subset of leaves is usually occupied. Empty subtrees are represented by publicly known default hashes. This default-hash mechanism makes SMTs a

common building block for revocation maps, authenticated dictionaries, and blockchain state commitments [11], [14], [15], [16], [17]. Despite the sparse representation, membership verification keeps the standard Merkle proof interface: a verifier checks a leaf value together with one sibling digest per level and recomputes the committed root.

The standard SMT proof API is efficient, but it reveals the queried target. When a client requests a membership proof, the server learns which key, account, certificate, or state object is being checked, and it also observes the corresponding authenticated path. Existing SMT work mainly improves storage, proof generation, multiproofs, and updates. These optimizations reduce the cost of proof serving, but they do not provide query privacy. This paper studies private retrieval of SMT membership proofs for a fixed public snapshot: the snapshot and its public metadata may be known, while the queried occupied leaf should remain hidden.

Private information retrieval (PIR) is the natural tool for hiding a database index, but an SMT membership proof is not a single database record. A standard proof is a height- h array of sibling digests. Some of these digests are non-default values that must be obtained from the server, while others are default hashes that the client can reconstruct locally from the public default-hash chain. Running one independent PIR query per proof level would hide the target, but it would treat a structured Merkle proof as h unrelated private lookups.

Thus, private SMT proof retrieval is better viewed as a structured batch-PIR problem. The client must retrieve several sibling digests using a target-independent query pattern, without revealing which Merkle path those digests belong to. Generic batch-PIR techniques can privately retrieve multiple records, but they treat the requested records as an arbitrary set. This line includes classical batch codes and combinatorial batch codes [29], [30], probabilistic batch codes (PBCs) as used in SealPIR [33], and recent multi-query PIR systems such as Vectorized BatchPIR and PIRANA [39], [40]. These systems improve the cryptographic or coding layer, but they do not decide which SMT proof elements should be stored in the PIR database, and they do not specify how the client should reconstruct the complete height- h proof using public default hashes.

Merkle proofs provide additional structure: all required sibling digests are determined by one root-to-leaf path. TreePIR [22] exploits this structure for perfect binary Merkle trees by coloring proof nodes so that each proof path con-

tains at most one node of each color, and then issuing one PIR query per color. This coloring principle is useful, but it does not directly apply to sparse SMT proof retrieval. TreePIR is formulated for a full perfect-tree proof layout. If an SMT is first expanded into a full tree, public default positions and inactive coordinates become part of the PIR database. If those default positions are removed afterward, the remaining records no longer have the full-tree structure assumed by TreePIR’s indexing argument.

The main challenge is therefore to identify the right PIR database for SMT membership proofs. SparseTreePIR stores only non-default sibling digests that can appear in membership proofs of occupied leaves. For each such digest, the setup records the interval of occupied leaves whose proofs require that digest. These intervals form a laminar family, which allows the records to be partitioned into color classes so that any membership proof needs at most one real record from each class.

Online retrieval then has a fixed query pattern. For every target leaf, the client sends one PIR query to each color subdatabase. If the target proof contains a non-default sibling digest in that color, the query retrieves the corresponding record. Otherwise, the client sends a dummy query to the same subdatabase. The server sees the same number of PIR queries for every target and cannot distinguish real queries from dummy queries under the security of the underlying PIR scheme. After receiving the responses, the client inserts the real digests into the appropriate proof levels and fills all remaining levels using the public default-hash chain. The result is an ordinary height- h SMT membership proof.

This formulation separates two issues that are often conflated. The first issue is storage: the PIR database should contain only non-default proof digests, not public default hashes or inactive logical coordinates. The second issue is query width: the number of PIR subqueries should depend on the maximum number of non-default sibling digests needed by any occupied-leaf proof, rather than on the logical SMT height. On sparse workloads, this number can be much smaller than h , while the final proof verified by the client still has length h .

A valid coloring, however, is not necessarily an efficient layout. Some color classes may contain many more records than others, and the largest subdatabase can dominate the cost of the PIR backend, especially when queries are executed in parallel or when the backend cost is sensitive to database size. SparseTreePIR therefore includes ActiveBalance, a load-balancing algorithm that applies subtree-level color permutations to reduce the largest color class while preserving the one-query-per-color retrieval pattern.

This paper makes the following contributions.

- 1) **SMT-specific PIR database construction.** We show that, for fixed-snapshot SMT membership proofs, the PIR database only needs to contain non-default sibling digests that can occur in proofs of occupied leaves. Public default hashes are reconstructed locally. The setup associates each stored digest with a target-

leaf rank interval, enabling client-side lookup without changing the standard SMT verification interface.

- 2) **Fixed-pattern batch retrieval.** We give a coloring-based retrieval layout in which each membership proof requires at most one real record from each color subdatabase. The client sends one PIR query per color and uses dummy queries when a color is not needed. This gives a target-independent batch-PIR transcript while preserving ordinary height- h SMT proof reconstruction.
- 3) **Balancing and evaluation.** We introduce ActiveBalance, a load-balancing algorithm for the color subdatabases. Experiments on SMT workloads compare SparseTreePIR with complete-tree TreePIR, height-preserving pruned TreePIR, a PBC-style SMT baseline, and unpartitioned active-record PIR. Repeated SimplePIR and PIANO runs show $2.1\times$ – $2.3\times$ lower online communication than the PBC-style baseline on the same snapshots, without modifying the underlying PIR primitive.

Figure 1 summarizes how SparseTreePIR transforms ordinary SMT proof serving into private proof retrieval.

The rest of the paper is organized as follows. Section 2 gives the background and problem setting needed for our formulation. Section 3 positions SparseTreePIR relative to authenticated maps, TreePIR, and PIR backends. Section 4 defines the problem, leakage model, and PIR-retrievable sibling nodes. Section 5 presents the SparseTreePIR construction. Section 6 proves correctness, target privacy, and complexity properties. Section 7 evaluates the resulting private-retrieval layout and balancing algorithm. Section 8 concludes.

2. Background and Problem Setting

This section reviews the background and problem setting needed for private retrieval of Sparse Merkle Tree (SMT) membership proofs. We use standard terminology where possible. The proof verified by the client remains a standard height- h SMT membership proof: one sibling digest per level, checked against a public root. The database queried through PIR should contain only the non-default proof material that cannot be reconstructed from public default hashes. The client combines PIR-retrieved digests with the public default-hash chain before ordinary verification.

2.1. Merkle Proof Retrieval

A binary Merkle tree commits to a set of records by hashing the records at the leaves and recursively hashing pairs of child digests until a single root digest is obtained [1]. To verify membership of a target leaf, the verifier uses the leaf value and an authentication path consisting of one sibling digest at each level from the leaf to the root. The verifier recomputes the path upward and accepts if the resulting root equals the public commitment.

From a retrieval perspective, a Merkle authentication path is a path-dependent batch of records rather than a

SparseTreePIR: Solving the SMT-to-Private-Retrieval Challenge

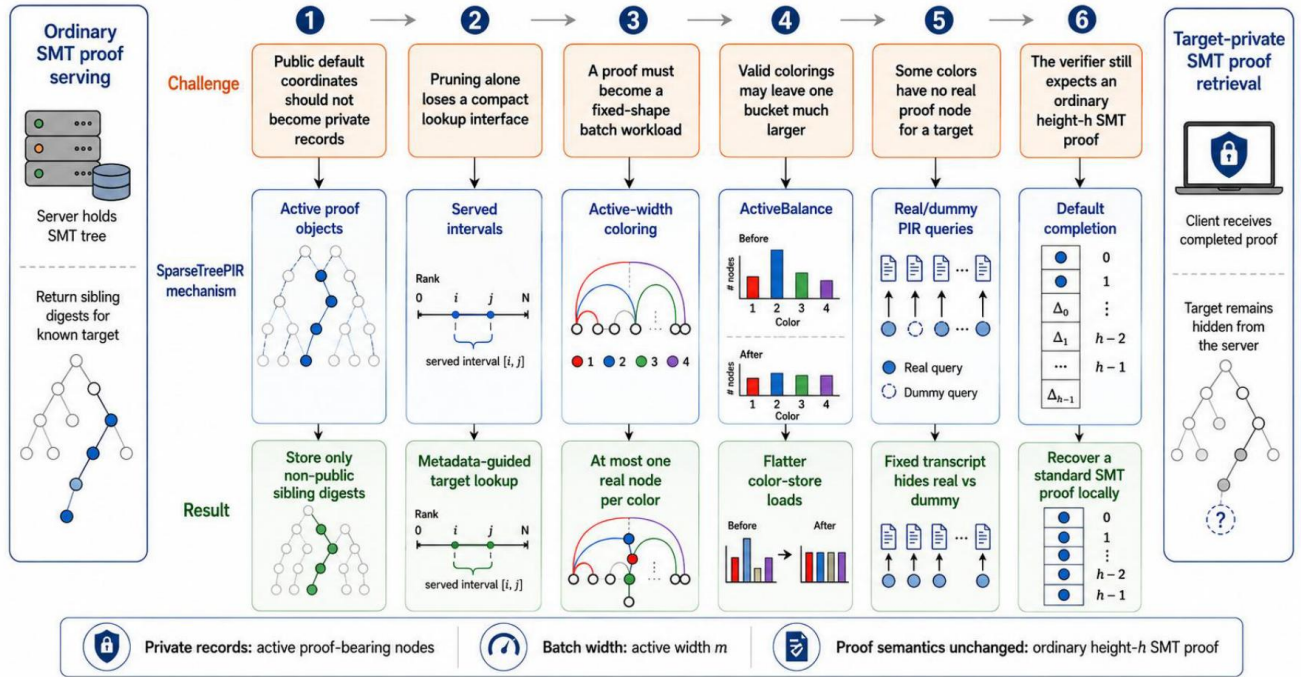


Figure 1. SparseTreePIR solution map from ordinary SMT proof serving to target-private proof retrieval. The retrieval mechanism stores only non-default proof digests in color subdatabases, uses target-interval metadata for lookup, hides real and dummy PIR roles with a fixed query pattern, balances subdatabase sizes with ActiveBalance, and reconstructs the standard height- h SMT proof locally.

single database item. A direct proof request reveals the queried leaf: when the client asks the server for the proof of coordinate i , the server learns both i and the corresponding path in the tree. PIR hides a selected database index, but applying PIR independently to every proof level ignores this path structure. TreePIR addresses the issue for perfect binary Merkle trees by partitioning proof nodes into color classes so that each proof path contains at most one node from each class, and the client sends one PIR query per class [22]. We follow the same path-aware batch-retrieval principle, but sparse SMTs require a different retrieval database.

2.2. Sparse Merkle Proofs and Default Hashes

A Sparse Merkle Tree is a Merkle tree over a fixed logical address space. A binary SMT of height h has 2^h possible leaf coordinates, although only a subset of those coordinates may be occupied. Empty subtrees are represented by public default digests. Let $\Delta_0, \Delta_1, \dots, \Delta_{h-1}$ denote the default-hash chain, where Δ_0 is the digest of an empty leaf and $\Delta_{t+1} = H(\Delta_t || \Delta_t)$. An empty sibling subtree of height t therefore has digest Δ_t .

An SMT membership proof for an occupied leaf still contains one sibling digest per level, so the proof length is always h . The important sparse-tree property is that some proof entries are public: if a sibling subtree is empty, the corresponding entry is the default digest for that level and does not need retrieval. If the sibling subtree is non-empty,

its digest is a non-default sibling digest that may have to be obtained from the server. Thus the verification view is the standard height- h membership proof, while the retrieval view contains only the non-default sibling digests that can appear in such proofs. The retrieval protocol changes how the proof is obtained, not how it is verified.

Figure 2 illustrates the two views of the same SMT snapshot. The verifier reasons about the full logical coordinate tree. The retrieval layer, however, should not materialize empty coordinates as PIR records: non-default nodes carry the only proof material that must be retrieved, while empty proof levels are filled from the public default-hash chain.

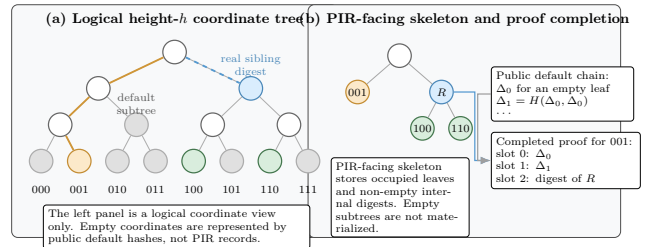


Figure 2. Logical and retrieval views of an SMT. The verifier reasons about a complete height- h coordinate tree, but the PIR database should not store every empty coordinate. A membership proof is completed by combining PIR-retrieved non-default sibling digests with public default hashes.

2.3. From Proof Privacy to Batch PIR

The privacy goal is query privacy for membership-proof retrieval: given a fixed public SMT snapshot, the server should not learn which occupied leaf the client is proving, nor which authentication path is selected. The tree root, the default-hash chain, and the public retrieval layout may be known to the server.

A simple solution would run one PIR query for every level of the height- h proof. This hides the target, but it treats the proof as h independent lookups and may issue unnecessary queries for levels whose sibling digests are public defaults. Generic batch-PIR schemes can retrieve multiple records privately, but they treat the requested records as an arbitrary batch. They do not specify which SMT proof records should be stored in the PIR database, how default proof entries should be omitted, or how the client should reconstruct the standard SMT proof after retrieval. The sparse setting therefore needs an SMT-specific retrieval layout: simply deleting empty records from a perfect-tree layout is not enough, because the remaining records must still be indexed, partitioned into subdatabases, queried with a fixed query pattern, and mapped back to the correct proof levels.

The construction in this paper stores only non-default sibling digests that may be required by some occupied-leaf membership proof. These records are partitioned into subdatabases so that, for any target, the client needs at most one real record from each subdatabase. The online protocol sends one PIR query to each subdatabase. If the target proof requires a record from that subdatabase, the query selects that record; otherwise, it is a dummy query sampled through the same PIR interface. The server observes a fixed-size batch of well-formed PIR queries, while the client locally determines which responses correspond to proof entries.

2.4. Retrieval Workload Metrics

We treat the underlying PIR scheme as a black-box backend. The contribution of the retrieval layout is therefore the database and query profile exposed to existing PIR schemes, rather than a new PIR primitive. For a fixed SMT snapshot, the main quantities are: (i) the number of non-default proof records stored in the PIR database; (ii) the number of PIR subqueries sent for one membership proof; (iii) the maximum subdatabase size; and (iv) the amount of public metadata needed for client-side lookup.

These quantities do not fully determine every PIR implementation’s running time, because practical systems differ in packing methods, preprocessing, cryptographic parameters, and query/response formats [26], [27], [32], [33], [35], [36]. They do determine the structural workload given to the backend: sequential execution scales with the queried subdatabases, while parallel execution is often limited by the largest subdatabase. This resource view also clarifies the baseline. Once query privacy is required, the relevant comparison is not ordinary SMT proof serving, but privacy-preserving layouts that store different proof-record sets,

issue different numbers of PIR subqueries, and expose different subdatabase sizes.

The resulting design problem is as follows. Given a fixed public SMT snapshot, determine which non-default sibling digests should be stored as PIR records, how these records should be partitioned into subdatabases, and how the client should reconstruct the standard height- h SMT membership proof from PIR responses and public default hashes. Section 4 formalizes the retrievable record set, the associated target intervals, and the leakage model. Section 5 then presents the SparseTreePIR construction.

3. Related Work

Merkle trees and authenticated data structures. Merkle’s hash-tree construction [1] underlies authenticated dictionaries and generic authenticated data structures [6], [41]. Deployed systems use Merkle-style commitments for certificate-transparency logs [2], [3], [4], key-transparency and verifiable-directory systems [7], [8], [9], [10], replicated key-value stores [5], append-only authenticated structures [42], and blockchain or stateless-client commitments [13], [14], [15]. These systems provide integrity and efficient verification, but they do not hide which proof a client requests. Our setting keeps the SMT snapshot and public metadata visible, while hiding the queried occupied leaf.

Sparse Merkle trees and proof generation. SMTs support large logical key spaces by assigning public default hashes to empty subtrees [16]. They appear in revocation/transparency maps, authenticated maps, constrained revocation checking, and state-commitment structures such as Jellyfish Merkle Tree [11], [12], [17]. Recent work optimizes proof size, multiproofs, non-membership proofs, batch updates, or alternative authenticated structures [18], [19], [21]. These systems improve ordinary SMT storage, updates, or proof generation. SparseTreePIR keeps the standard verification interface: the client reconstructs a height- h membership proof. The change is at the retrieval layer, where only non-default sibling digests are queried through PIR and default-empty levels are reconstructed locally.

Private retrieval of authenticated proofs. Private retrieval of authenticated proofs has been studied in multi-client and transparency-log settings [23], [24], [25]. The closest structural baseline is TreePIR [22], which partitions proof nodes of a perfect binary Merkle tree into color classes so that each proof path contains at most one node from each class. SparseTreePIR uses the same one-query-per-color principle, but the database records are different. TreePIR is formulated for a perfect-tree proof layout; an SMT contains public default proof positions that should not be stored or queried through PIR. Pruning those records after expanding to a perfect tree still leaves the wrong query universe. SparseTreePIR instead builds the PIR database directly from non-default sibling digests that can occur in occupied-leaf membership proofs, uses interval metadata for lookup, and reconstructs the ordinary height- h proof with

TABLE 1. REPRESENTATIVE AUTHENTICATED-MAP AND SMT PROOF-SERVING SYSTEMS AND THEIR RELATION TO PRIVATE PROOF RETRIEVAL

Work	Reported goal/result	Optimized layer	Relation to our setting
Efficient SMTs [16]	Formal SMT operations and caching; audit paths for membership/non-membership can be generated in practically constant time, reported below 4 ms with SHA-512/256	Plain proof generation	Non-private lower bound: server knows the queried coordinate
Jellyfish Merkle Tree [17]	SMT variant for Diem state storage; optimizes node keys, node types, proof format, LSM-tree storage, I/O, and computation	Deployed state-tree storage	Representative SMT-system baseline, but proof requests are visible lookups
Compact Merkle multiproofs [18]	Sparse multiproof encoding keeps only k leaf indices instead of an index for every non-leaf hash in a multiproof	Multi-target proof compression	Reduces proof payload for known targets; does not hide which targets are requested
Rollup SMT batch update [19]	For zkSync Lite-style updates, improves account update time from $O(\log n)$ to $O(1)$ and reduces batch update cost by a one-pass traversal	State update/root recomputation	Orthogonal to retrieval privacy; update cost is not the bottleneck studied here
Multicollision-resistant non-membership tree [21]	Reduces authenticated (non-)membership tree depth for ZK-oriented settings while preserving security parameters	Alternative accumulator/proof structure	Changes the authenticated structure; our work keeps standard SMT proof semantics while privatizing retrieval

default hashes. Thus it is an SMT proof-retrieval layout, not a new PIR primitive and not a direct TreePIR instantiation.

Batch PIR and practical PIR backends. PIR began with multi-server information-theoretic constructions [26] and single-server computational PIR [27], [28]. Batch retrieval uses batch codes and combinatorial batch codes [29], [30]; practical systems use probabilistic batch codes, Cuckoo-style placement, vectorized processing, or constant-weight-code techniques [31], [33], [39], [40]. Backends such as XPIR, OnionPIR, Spiral, SimplePIR, PIANO, and YPIR improve communication, server computation, packing, and preprocessing [32], [34], [35], [36], [37], [38]. These systems provide the private-retrieval backend. They do not specify which SMT proof records should enter the PIR database, how to group them into subdatabases, or how to reconstruct the standard SMT proof. SparseTreePIR exposes such a database layout to any compatible backend; real and dummy queries are hidden by the query privacy of the underlying PIR scheme.

Baselines derived from prior work. Existing SMT systems optimize ordinary proof serving: the server traverses the requested coordinate and returns the sibling digests needed for verification [11], [12], [16], [17]. Some default-empty siblings can be omitted because the client reconstructs them from public defaults, but the request still reveals the key or path. Table 2 separates what each prior line provides from what remains missing for batch PIR. Once query privacy is required, the relevant comparison is between PIR database layouts for the same proof-retrieval task, rather than only against non-private proof-serving systems or internal ablations.

Balancing color subdatabases. A valid coloring does not necessarily give an efficient PIR layout: one color subdatabase may be much larger than the others. Perfect-tree coloring reduces the number of queries for perfect Merkle trees, but it does not address the load imbalance caused by sparse SMT snapshots. ActiveBalance is a backend-independent refinement that preserves the same fixed number of PIR queries while reducing the largest queried sub-

database, complementing ordinary SMT proof-generation optimizations.

4. Problem Formulation and Leakage Model

This section formalizes the private retrieval problem studied in this paper. We consider a fixed public snapshot of a binary Sparse Merkle Tree (SMT). The tree height, root commitment, occupied-leaf set, public default-hash chain, and the PIR database layout produced during setup are treated as public snapshot information. The privacy goal is query privacy within this fixed snapshot: during online retrieval, the storage server should not learn which occupied leaf is queried, which non-default sibling digests are needed for its proof, or which PIR queries are real rather than dummy queries.

The formal model targets membership-proof retrieval for occupied leaves. Non-membership retrieval over absent coordinates is a natural extension, but it changes the query domain from the public occupied-leaf set to the full logical address space. It also requires metadata for absent coordinate regions and an explicit leakage choice about whether the server learns that the query is a non-membership query. We therefore leave non-membership privacy outside the present model.

A PIR record in this paper means a database record that is retrieved through PIR because it cannot be reconstructed from public default hashes. The record content itself is not assumed to be hidden from the storage server. The protected information is the selected index, or equivalently the queried target leaf, relative to the setup leakage defined below.

4.1. Fixed-Snapshot Proof Records

Let T be a fixed binary Sparse Merkle Tree of height h , and let $\mathcal{L}$ denote its set of occupied leaves. Hashes are still computed in the standard bottom-up way. The retrieval layout introduced below does not change the SMT hash construction; it only specifies which proof digests are placed

TABLE 2. PIR DATABASE LAYOUTS FOR PRIVATE SMT PROOF RETRIEVAL DERIVED FROM PRIOR WORK

Prior line	Natural proof object	Induced baseline	Missing for our goal
Standard SMT proof APIs [11], [16], [17]	Height- h proof path with non-default siblings and public defaults	Plain SMT proof serving	No target privacy; server sees the queried coordinate/path
Compact SMT storage [12], [16], [17]	Non-default skeleton plus public default chain	Non-default proof-record serving	Saves storage, but gives no fixed PIR query pattern
Generic batch-PIR placement [29], [30], [33]	Non-default proof records treated as an arbitrary batch	PBC-style SMT; unpartitioned active-record PIR	Ignores ancestor/path structure; may query or replicate too much
TreePIR on perfect trees [22]	Colored proof nodes in a perfect-tree layout	Complete-tree and height-preserving pruned TreePIR	Colors the full coordinate tree, including public default positions
This paper	Non-default sibling digests with interval metadata	SparseTreePIR	Fixed-pattern batch PIR with local default-hash reconstruction

in PIR subdatabases. We assume the standard public default-hash chain

$$\Delta_0, \Delta_1, \dots, \Delta_{h-1},$$

where Δ_0 is the default empty-leaf digest and $\Delta_{t+1} = H(\Delta_t \| \Delta_t)$ for $0 \leq t < h - 1$. Thus, every empty sibling subtree of height t has publicly reconstructible digest Δ_t .

Definition 4.1 (PIR-retrievable sibling node). *For a node u in T , let $\text{cnt}(u)$ be the number of occupied leaves in the subtree rooted at u . Let u be a non-root node, and let $\text{sib}(u)$ be its sibling. We say that u is a PIR-retrievable sibling node if*

$$\text{cnt}(u) > 0 \quad \text{and} \quad \text{cnt}(\text{sib}(u)) > 0.$$

The set of all PIR-retrievable sibling nodes is denoted by $\mathcal{A}(T)$.

The first condition ensures that u itself has a non-default digest. The second condition ensures that u can appear as a sibling node in the membership proof of at least one occupied target leaf. Thus $\mathcal{A}(T)$ contains exactly the non-default sibling digests that may have to be retrieved through PIR. Empty sibling subtrees remain outside the PIR database because their digests are publicly reconstructible from the default-hash chain. We sometimes call nodes in $\mathcal{A}(T)$ *active proof nodes*; this is only shorthand for PIR-retrievable non-default sibling nodes, not for arbitrary non-empty SMT nodes.

Proposition 4.1 (Minimal non-default PIR record set). *Fix a public height- h SMT snapshot T and the public default-hash chain, and assume the SMT hash function is collision resistant. In the record-based retrieval model where a PIR record supplies one proof sibling digest and public default values are filled locally, $\mathcal{A}(T)$ is exactly the set of non-default sibling digests that may be required by some occupied-leaf membership proof, except with negligible hash-collision probability. Consequently, any PIR database that supports all occupied-leaf membership proofs must make one digest for every node in $\mathcal{A}(T)$ retrievable, while no default-empty sibling position needs a PIR record.*

Proof. If $u \in \mathcal{A}(T)$, then u is non-empty and $\text{sib}(u)$ is also non-empty. Choose any occupied leaf in the subtree of $\text{sib}(u)$; the standard membership proof for that leaf contains

the digest of u as a sibling digest. By collision resistance, a non-empty subtree digest is computationally distinct from the corresponding public default digest except with negligible probability. Thus a record-based private retrieval layout that reconstructs every occupied-leaf proof must make this digest retrievable. Conversely, if a proof sibling subtree is empty, its digest is the public value Δ_t for the corresponding height t and is reconstructed locally. A non-empty node whose sibling subtree is empty is not a sibling for any occupied target leaf. Therefore the necessary and sufficient non-default PIR record set is precisely $\mathcal{A}(T)$.

Definition 4.2 (PIR-retrievable sibling set). *For any occupied leaf $\ell \in \mathcal{L}$, let*

$$\text{Path}(\ell) \subseteq \mathcal{A}(T)$$

denote the sequence of all PIR-retrievable sibling nodes in the standard Merkle proof of ℓ , ordered from higher to lower proof levels.

Equivalently, $\text{Path}(\ell)$ contains exactly those sibling nodes on the standard height- h proof of ℓ whose subtrees contain at least one occupied leaf. Sibling levels whose subtrees are empty are not PIR records and are later filled locally with the corresponding default hash.

Although the full SMT proof length is always h , the number of PIR records in $\text{Path}(\ell)$ may be smaller than h , because some levels correspond only to publicly reconstructible default-empty nodes.

Let

$$m = \max_{\ell \in \mathcal{L}} |\text{Path}(\ell)|$$

be the *retrieval width* of the snapshot, namely the maximum number of non-default sibling digests that can appear in one occupied-leaf membership proof. Equivalently, this is the active width used by SparseTreePIR. By construction, $m \leq h$. Once the PIR-retrievable record set is fixed, this same quantity is also the number of color classes and the batch width of the one-query-per-color retrieval layout. Throughout the paper, $[m]$ denotes $\{1, \dots, m\}$.

The case $|\mathcal{L}| = 1$ is degenerate: no occupied target has a non-default sibling subtree, so $\mathcal{A}(T) = \emptyset$, $m = 0$, and no PIR query is issued. Unless explicitly stated otherwise, the rest of the paper assumes $|\mathcal{L}| \geq 2$.

The role of m is analogous to the role of h in TreePIR only after public default-empty levels have been removed from the PIR database layout. It is not a tunable parameter chosen by the balancing algorithm; it is a public resource parameter of the fixed SMT snapshot. The verifier still receives a height- h proof after client-side reconstruction. The retrieval width only measures how many proof positions may require private retrieval under the record-based retrieval model.

Remark 4.1 (Why m can be much smaller than h). *The value of m is necessarily instance-dependent because it depends on where the occupied leaves branch. The setup phase publishes m as part of the public layout, just as a perfect-tree TreePIR setup publishes height h . To interpret this parameter, suppose n occupied leaves are sampled uniformly from a logical universe of size $U = 2^h$, and condition on one target leaf. At proof level t above the leaf, the sibling subtree contains 2^t logical leaves, so the probability that this level contributes a non-default sibling digest is*

$$1 - \frac{\binom{U-1-2^t}{n-1}}{\binom{U-1}{n-1}} \approx 1 - \exp\left(-\frac{(n-1)2^t}{U-1}\right).$$

Thus the expected number of PIR-retrievable proof positions for a random target is the sum of these probabilities over $t = 0, \dots, h-1$. When $n \ll U$, most low levels have empty sibling subtrees and only the levels near the branching depth of the occupied set contribute; the retrieval width therefore tracks the branching structure of the occupied leaves rather than the logical height. In adversarial layouts, the sharp worst-case bound remains $m \leq \min\{h, n-1\}$.

This paper focuses on target-private retrieval of membership proofs for occupied leaves in a fixed public SMT snapshot. This scope matches the proof-retrieval setting in which the client privately asks for the authentication material of an existing committed object. Non-membership retrieval is a natural extension, but it changes the object being hidden. The target domain becomes the full logical coordinate space rather than the public occupied-leaf set; a proof may certify an empty coordinate, a divergence from the nearest occupied prefix, or a gap between existing keys; and the protocol must specify whether the server should learn that the query is for an absent coordinate. Consequently, the interval metadata would have to range over logical coordinates or absent-coordinate regions, not only over occupied-leaf ranks. We therefore keep non-membership SparseTreePIR outside the present formal model and treat it as a separate extension requiring its own metadata and leakage model.

4.2. Target-Rank Intervals

To support efficient indexing, we associate each PIR-retrievable sibling node with the interval of target leaves for which it acts as a proof sibling. This is not the subtree interval of u itself. Order the occupied leaves by their logical SMT positions and let $r(\ell) \in \{0, \dots, |\mathcal{L}| - 1\}$ be the

occupied-leaf rank of ℓ . Let $s = \text{sib}(u)$, and define the *target-rank interval*

$$I_{\text{srv}}(u) = [L(s), R(s)]$$

as the interval of occupied-leaf ranks covered by the sibling subtree of u . Thus $L(s)$ and $R(s)$ are rank endpoints, not heap-index endpoints in the full logical coordinate space. Then, for any occupied leaf ℓ ,

$$u \in \text{Path}(\ell) \iff r(\ell) \in I_{\text{srv}}(u).$$

Since every binary subtree corresponds to a contiguous interval in leaf order, the resulting target-rank intervals form a laminar family: any two intervals are either disjoint, or one contains the other. This structure underlies both the coloring method and the indexing method. The intervals are public lookup metadata derived from the fixed snapshot and the public occupied-leaf order. Publishing them is therefore part of the setup leakage below; it does not attempt to hide the occupancy pattern or density of the snapshot. If the occupied-leaf set itself were sensitive, the layout would need additional occupancy-hiding mechanisms, such as dummy occupied leaves or oblivious metadata publication, which are outside the present query-privacy model.

4.3. Coloring and Load Metrics

The constructions below use m color classes, where m is the retrieval width defined above. Since $m \leq h$, the resulting batch profile is no wider than the SMT verifier height. Section 6 later states the corresponding model-specific lower-bound argument: under the one-query-per-color model, a proof requiring m PIR records needs m distinct colors.

Definition 4.3 (Proper coloring). *A map*

$$\varphi : \mathcal{A}(T) \rightarrow [m]$$

is a proper coloring if for every occupied leaf $\ell \in \mathcal{L}$ and every two distinct nodes $u, v \in \text{Path}(\ell)$, we have

$$u \neq v \implies \varphi(u) \neq \varphi(v).$$

A proper coloring ensures that every membership proof needs at most one real PIR record from each color subdatabase.

For a color $c \in [m]$, let

$$\mathcal{D}_c = \{u \in \mathcal{A}(T) : \varphi(u) = c\}$$

be the corresponding color subdatabase, with size $N_c = |\mathcal{D}_c|$. Let $N = |\mathcal{A}(T)|$, and let

$$\bar{N} = N/m$$

be the average subdatabase size. The main optimization target in this paper is to keep the color subdatabases as balanced as possible while preserving proper coloring. In particular, we track

$$M(\varphi) = \max_c N_c, \quad \Delta_N(\varphi) = \max_c N_c - \min_c N_c,$$

and the normalized maximum-subdatabase ratio

$$R(\varphi) = \frac{M(\varphi)}{\bar{N}}.$$

Here $M(\varphi)$ is the direct structural bottleneck, while $\Delta_N(\varphi)$ and $R(\varphi)$ indicate how far the partition is from an equal split.

The ActiveBalance scheme is self-contained. It first constructs a feasible coloring by a first-fit traversal of the laminar interval forest, and then refines that coloring by subtree-level color permutations that directly improve the global subdatabase loads.

Definition 4.4 (Client-side inputs and public metadata). *For a target leaf ℓ , the client uses the following local inputs and public metadata before issuing PIR queries:*

- 1) the tree height h and the position of ℓ in the SMT;
- 2) the default-hash chain $\Delta_0, \dots, \Delta_{h-1}$;
- 3) the occupied-leaf rank $r(\ell)$ of the target within the public ordered set of occupied leaves;
- 4) for each color $c \in [m]$, a public metadata table

$$\mathcal{M}_c = ((I_{\text{srv}}(u), \delta(u)))_{u \in \mathcal{D}_c},$$

sorted by interval, where the two endpoints of $I_{\text{srv}}(u)$ are occupied-leaf ranks and $\delta(u) = h - \text{depth}(u)$ denotes the proof level of u measured from leaf to root.

The PIR subdatabases store node digests. The level and interval information required for client-side proof reconstruction is carried by the public metadata tables rather than by PIR records.

4.4. Setup Leakage and Target Privacy

The retrieval protocol in this paper hides the queried target leaf inside one *fixed public SMT instance*; it does not aim to hide the entire occupancy pattern of that instance. We model the storage server’s snapshot-level view with the following deterministic setup leakage.

We assume the underlying backend is a correct single-item PIR scheme with query privacy over each fixed subdatabase: for any two valid indices in the same subdatabase, the server-observable query distributions are computationally indistinguishable. We also require the real and dummy queries for a given subdatabase to use the same PIR query interface and the same observable query/response lengths. Otherwise, a backend-level length or format difference could reveal whether a color is real or dummy.

Definition 4.5 (Setup leakage). *Let setup on a fixed SMT snapshot T produce retrieval width m , PIR-retrievable node set $\mathcal{A}(T)$, coloring $\varphi : \mathcal{A}(T) \rightarrow [m]$, color subdatabases $\{\mathcal{D}_c\}_{c=1}^m$, and public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$. Let $N_c = |\mathcal{D}_c|$. The setup leakage visible to the storage server is*

$$\mathcal{L}_{\text{setup}}(T) = \left(h, \text{root}(T), \mathcal{L}, \{\Delta_t\}_{t=0}^{h-1}, \mathcal{A}(T), m, \varphi, \{(\mathcal{D}_c, \mathcal{M}_c, N_c)\}_{c=1}^m \right).$$

TABLE 3. SERVER-VISIBLE SETUP STATE AND HIDDEN QUERY INFORMATION

Visible to the storage server	Hidden by PIR queries
root, occupied-leaf set, default-hash chain, PIR-retrievable node set, color subdatabases, interval metadata, color assignment, subdatabase sizes, retrieval width m	target leaf ℓ , required PIR records, which color slots are real or dummy, selected item index inside each color subdatabase

Thus the leakage includes the fixed occupancy pattern, the committed root, the public default-hash chain, the PIR-retrievable node set, the coloring, the color subdatabases, the public metadata, and the subdatabase sizes. For an online target ℓ , it does not include ℓ , the set $\text{Path}(\ell)$ of required PIR records, the set of colors containing real proof records, or the selected item index inside any color subdatabase. The only target-independent online shape revealed by the protocol is that every retrieval sends exactly one PIR subquery to each of the m color subdatabases.

The hidden query information consists of the target leaf ℓ , equivalently the required PIR record set $\text{Path}(\ell)$, as well as which color positions correspond to real retrievals rather than dummy subqueries and which item index is privately selected inside each color subdatabase.

We make this leakage profile explicit. The retrieval layout is precomputed once for a fixed SMT snapshot and is treated as server-visible structure thereafter. Our privacy claim is therefore *query privacy relative to a server-visible PIR database layout*, rather than structure privacy for the whole sparse tree. The corresponding transcript view has a fixed query shape: the server sees one well-formed PIR query per color, while the real/dummy roles are client-local.

The privacy model is honest-but-curious with respect to the storage server. A malicious server cannot make an incorrect digest verify against the public SMT root except with the usual hash-collision probability, but it can cause retrieval failure or equivocate about published metadata unless the subdatabases and metadata are authenticated. Such authenticated publication, for example by committing to the public layout or by using an authenticated PIR service layer, is complementary to the query-privacy mechanism studied here.

Definition 4.6 (Target privacy game). *Let $\mathcal{L}_{\text{setup}}(T)$ be the setup leakage in Definition 4.5. In the experiment $\text{PrivTarget}_{\mathcal{A}}(\lambda)$, the challenger runs setup for the fixed SMT snapshot and gives $\mathcal{L}_{\text{setup}}(T)$ to the adversary $\mathcal{A}$. The adversary chooses two occupied target leaves $\ell_0, \ell_1 \in \mathcal{L}$. The challenger samples $b \leftarrow \{0, 1\}$, runs the batch query-generation algorithm for ℓ_b , and gives the full batch query transcript to $\mathcal{A}$. The adversary outputs a bit b' . Its advantage is*

$$\text{Adv}_{\mathcal{A}}^{\text{target}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

The protocol is target-private relative to $\mathcal{L}_{\text{setup}}(T)$ if this advantage is negligible for every efficient adversary.

Definition 4.7 (Dummy query distribution). *For a non-empty color subdatabase $\mathcal{D}_c$ of size N_c , a dummy query samples an index $d_c \leftarrow \{0, \dots, N_c - 1\}$ independently of the target leaf and invokes the same query algorithm $\text{PIR.Query}(\mathcal{D}_c, d_c)$ used for real records.*

For dummy positions, the client uses Definition 4.7 and discards the response locally. The setup layout exposes only non-empty color subdatabases. Implementations that allow empty color classes can remove them during setup or pad them with one public dummy record.

5. SparseTreePIR Construction

This section presents the fixed-snapshot retrieval protocol. Offline preprocessing constructs the PIR database from the compressed non-default SMT representation, rather than from a materialized full binary tree. Online retrieval issues a target-independent batch of PIR queries and reconstructs the standard height- h SMT membership proof locally using public default hashes.

5.1. Construction Overview

SparseTreePIR instantiates a snapshot-specific retrieval layout with a target-independent online transcript. During setup, the server identifies active proof records, partitions them into color subdatabases, and publishes metadata that maps target ranks to lookup intervals. During online retrieval, the client sends exactly one PIR subquery to each color subdatabase. If the target proof needs a record in that color, the query selects that record. Otherwise, the query is a dummy sampled through the same PIR interface. After receiving the responses, the client writes the real digests into the appropriate proof slots and fills the remaining slots with public default hashes. Figure 3 summarizes this setup-and-retrieval flow.

Online retrieval pattern. Given a fixed public SMT snapshot T , setup outputs color subdatabases $\mathcal{D}_1, \dots, \mathcal{D}_m$, public metadata tables $\mathcal{M}_1, \dots, \mathcal{M}_m$, and m color classes. For every occupied target leaf, the client sends exactly one PIR subquery to each $\mathcal{D}_c$. A subquery is real if color c contains the non-default sibling digest required by the target proof, and dummy otherwise. The client then combines the real PIR outputs with the public default-empty hash chain to reconstruct the standard height- h SMT proof. The PIR workload metrics are storage $|\mathcal{A}(T)|$, retrieval width m , largest subdatabase size $\max_c |\mathcal{D}_c|$, and public metadata size $O(|\mathcal{A}(T)|)$.

The algorithms below are presented as a protocol stack rather than as independent routines. Algorithm 1 is the server-side setup procedure for one fixed public SMT snapshot. It extracts active proof records, computes the retrieval-width parameter, invokes Algorithm 2 as a balancing subroutine, and publishes the metadata consumed by the client. Algorithm 3 is the online client lookup and query-generation procedure. It consumes exactly the public outputs of setup and produces two outputs: the batch of PIR subqueries sent

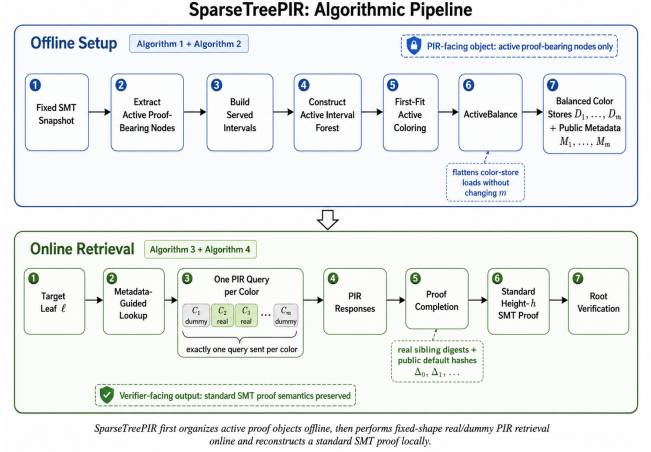


Figure 3. SparseTreePIR algorithmic pipeline. Offline setup extracts active proof records, builds target-rank intervals, colors the laminar interval forest, and balances color subdatabases. Online retrieval uses public metadata to issue exactly one PIR query per color, then combines retrieved non-default sibling digests with public default hashes to reconstruct the standard height- h SMT proof.

to the server, and a client-side selection map that is never revealed. After the underlying PIR backend returns one response per color, Algorithm 4 uses that selection map, the public metadata, and the default-empty hash chain to reconstruct the standard height- h SMT proof.

This separation mirrors the presentation style of TreePIR: setup fixes the colored database layout once, and every online retrieval has the same external profile. The server observes one PIR query per color subdatabase. Only the client knows which queries retrieve proof records and which are dummy queries.

5.2. Running Example

Figure 4 presents the running example used in this section. It shows a complete height-4 coordinate tree for explanation, but the implementation traverses the compressed non-default representation and stores only active proof records in color subdatabases. Because the model has already defined the record set, target-rank intervals, and valid coloring, the example can now be interpreted as an instance of the formal interface rather than as an additional rule.

The example separates three standard objects. The first is the *full SMT coordinate tree*, which determines hash positions and proof levels. The second is the set of non-default sibling digests that may appear in membership proofs; these digests form the active proof records stored in the PIR database. The third is *client-side proof reconstruction*: proof levels whose sibling subtrees are empty are filled with public default hashes rather than retrieved through PIR. The figure labels intervals using logical leaf identifiers for readability; the actual public metadata stores target-rank intervals over the ordered occupied leaves.

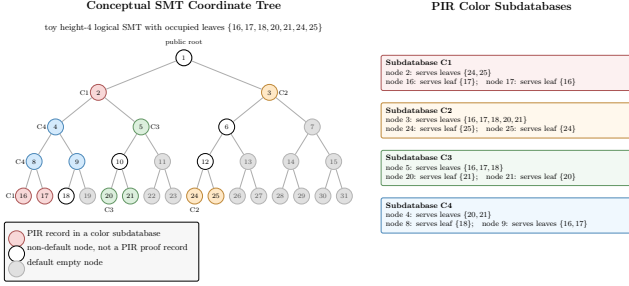


Figure 4. Running SMT example and its final color subdatabases. The full coordinate tree is explanatory; only active proof records are stored in PIR subdatabases.

The occupied leaves are $\{16, 17, 18, 20, 21, 24, 25\}$. The active proof records are exactly the twelve nodes shown in the color subdatabases of Figure 4:

$$\{2, 3, 4, 5, 8, 9, 16, 17, 20, 21, 24, 25\}.$$

The longest PIR-retrievable sibling set has size 4, so the setup exposes four color classes in this example.

This instance is useful because it already exhibits the balance problem. The first-fit feasibility stage may produce the legal color split

$$\begin{aligned} \mathcal{D}_1 &= \{3, 2\}, & \mathcal{D}_2 &= \{5, 4, 25, 24\}, \\ \mathcal{D}_3 &= \{9, 8, 21, 20\}, & \mathcal{D}_4 &= \{17, 16\}, \end{aligned}$$

whose subdatabase sizes are $(2, 4, 4, 2)$. The coloring is valid but imbalanced. By contrast, subtree recoloring can transform the same interval forest into

$$\begin{aligned} \mathcal{D}_1 &= \{2, 17, 16\}, & \mathcal{D}_2 &= \{3, 25, 24\}, \\ \mathcal{D}_3 &= \{5, 21, 20\}, & \mathcal{D}_4 &= \{4, 9, 8\}, \end{aligned}$$

which is still valid and now has balanced subdatabase sizes $(3, 3, 3, 3)$. This final split is deliberately not a layer coloring: for example, nodes 4, 8, and 9 share color 4 even though they lie at different logical depths in the full coordinate tree, while depth-1 nodes 2 and 3 have different colors. This is possible because the constraint is induced by target-rank intervals, not by raw tree depth. Figure 5 visualizes this change. In the current algorithm, this improvement is reached by a short sequence of accepted legal subtree recolorings; the online query pattern stays fixed throughout. We will reuse the same small instance again when illustrating one concrete query and client-side proof reconstruction.

5.3. Offline Preprocessing

The preprocessing algorithm is run once for a fixed public SMT snapshot. It takes the occupied leaf set and the compressed non-default SMT representation as input, and outputs the color subdatabases and public metadata used by online retrieval. It does not materialize the full logical tree. Instead, it traverses the compressed non-default structure, identifies the non-default sibling digests that may appear in

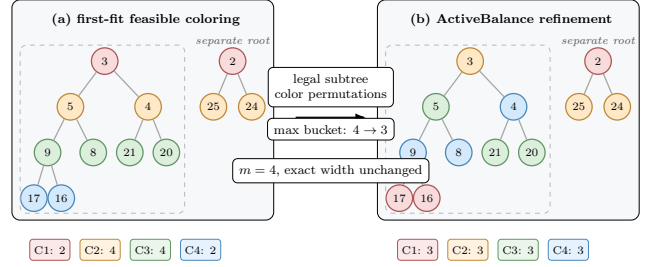


Figure 5. Running example for the coloring stage. The first-fit feasibility stage leaves an uneven but valid split, while ActiveBalance applies accepted legal subtree recolorings and reaches a flatter partition on the same interval forest.

membership proofs, and records for each digest its target-rank interval and proof level. The resulting intervals form a laminar interval forest. Setup then computes a valid coloring, applies ActiveBalance to reduce the largest subdatabase size, and publishes the metadata tables.

Algorithm 1 Offline preprocessing

Input: Fixed public SMT snapshot T , occupied leaf set $\mathcal{L}$, balancing rounds R_1
Output: Active width m , color subdatabases $\{\mathcal{D}_c\}_{c=1}^m$, public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$, valid coloring φ

- 1: Sort the occupied leaves by logical SMT position and assign occupied-leaf ranks $0, \dots, |\mathcal{L}| - 1$
- 2: Build the compressed non-default skeleton $\mathcal{S}(T)$ induced by the occupied leaves
- 3: Annotate each skeleton edge with its corresponding original SMT child root and skipped logical levels
- 4: Initialize $\mathcal{A}(T) \leftarrow \emptyset$
- 5: **for** each branching skeleton node p with two non-default original children **do**
- 6: **for** each original child root u of p **do**
- 7: $s \leftarrow \text{sib}(u)$ in the full SMT coordinate system
- 8: Add u to $\mathcal{A}(T)$
- 9: Assign target-rank interval $I_{\text{srv}}(u) = [L(s), R(s)]$ and proof level $\delta(u)$
- 10: **end for**
- 11: **end for**
- 12: Sort all target-rank intervals by increasing left endpoint and decreasing interval length
- 13: Build the interval forest $\mathcal{F}$ from interval-containment relations
- 14: Set m to the maximum depth of $\mathcal{F}$
- 15: Compute $\varphi \leftarrow \text{FIRSTFITCOLORING}(\mathcal{F}, m)$
- 16: Refine $\varphi \leftarrow \text{ActiveBalance}(\mathcal{F}, \varphi, R_1)$ using Algorithm 2
- 17: Build the color subdatabases $\mathcal{D}_c$ and the public metadata tables $\mathcal{M}_c$
- 18: **return** $m, \{\mathcal{D}_c\}_{c=1}^m, \{\mathcal{M}_c\}_{c=1}^m, \varphi$

The first-fit stage is only a feasibility step. It traverses each tree of $\mathcal{F}$ from root to leaves and assigns every node the smallest color in $[m]$ not used on its strict ancestor chain. Since no target proof requires more than m PIR records, such a color always exists. The actual balancing contribution is the subsequent ActiveBalance recoloring step. The protocol-complete formulation remains specific to fixed binary SMTs, because client-side proof reconstruction relies on the standard default-empty hash chain.

From a systems viewpoint, Algorithm 1 is the offline setup stage of the construction. Once the color subdatabases and public metadata tables have been built for one fixed

Algorithm 2 ActiveBalance subtree recoloring

Input: Laminar interval forest $\mathcal{F}$, valid coloring φ , rounds R_1
Output: Updated valid coloring φ

- 1: Compute global loads $\{N_c\}_{c=1}^m$ and $\Psi^* \leftarrow \Psi(\varphi)$
- 2: **for** $t = 1$ **to** R_1 **do**
- 3: Annotate each subtree root U with ancestor colors and histogram $\{C_U(c)\}_{c=1}^m$
- 4: $\text{best} \leftarrow \text{NONE}$
- 5: **for** each subtree root U in $\mathcal{F}$ **do**
- 6: $A(U) \leftarrow [m] \setminus \text{AncestorColors}(U)$
- 7: **if** $|A(U)| \leq 1$ **then**
- 8: **continue**
- 9: **end if**
- 10: Build $K_U[a, b] = (N_b - C_U(b) + C_U(a))^2$ for $a, b \in A(U)$
- 11: Let π_U be the min-cost perfect matching on K_U
- 12: Form φ' by applying π_U inside U and leaving all nodes outside U unchanged
- 13: **if** $\Psi(\varphi') < \Psi^*$ **then**
- 14: $(\text{best}, \Psi^*) \leftarrow ((U, \pi_U, \varphi'), \Psi(\varphi'))$
- 15: **end if**
- 16: **end for**
- 17: **if** $\text{best} = \text{NONE}$ **then**
- 18: **break**
- 19: **end if**
- 20: Apply best and update φ , $\{N_c\}_{c=1}^m$, and Ψ^*
- 21: **end for**
- 22: **return** φ

SMT instance, the online retrieval protocol never needs to see the whole tree again. The expensive structural work is pushed offline, while the online stage sees only interval tables, color subdatabases, and the public default-hash chain.

5.4. ActiveBalance: Load Balancing by Subtree Recoloring

A valid coloring guarantees that each proof needs at most one real record from each color subdatabase, but it does not guarantee balanced subdatabase sizes. A highly imbalanced coloring can make one PIR query much more expensive than the others. ActiveBalance is a load-balancing procedure that preserves the retrieval width m and changes only the assignment of records to colors. It evaluates subtree recolorings using the lexicographic load vector

$$\Psi(\varphi) = \text{sort}_{\downarrow}(N_1, \dots, N_m),$$

so the objective is to reduce the largest subdatabase first and then flatten the remaining count vector.

For a rooted subtree U of the interval forest and a color $c \in [m]$, let $C_U(c)$ denote the number of nodes of color c currently appearing inside U . Let $A(U) \subseteq [m]$ be the set of colors not used on the strict ancestor chain of U . Since every node in U must already be colored from $A(U)$, any permutation π_U of these available colors can be applied uniformly inside U without violating the coloring constraint. Proposition 6.1 shows that every such permutation preserves validity.

Algorithm 2 makes two design choices explicit. First, each subtree is evaluated against a count-only quadratic assignment objective rather than against the full combinatorial search over all possible recolorings. The cost term

$(N_b - C_U(b) + C_U(a))^2$ is the squared size of subdatabase b after the portion of color b currently inside U is removed and the portion of local color a is mapped into it. Thus the surrogate favors moving heavier local color mass toward lighter global subdatabases. Second, after solving that local assignment, we accept a move only if the true global signature $\Psi(\varphi)$ decreases. Hence the surrogate guides the search, while the lexicographic load vector remains the actual acceptance rule.

5.5. Target-Independent Online Query Generation

After preprocessing, the client uses only public metadata to determine which color subdatabases contain proof records for the target. For each color c , the metadata table $\mathcal{M}_c$ is sorted by target-rank interval. Since intervals with the same color are disjoint, one binary search returns either a unique matching record or no matching record.

Algorithm 3 Online query generation: one PIR subquery per color

Input: Target leaf ℓ with occupied-leaf rank $r(\ell)$, color subdatabases $\{\mathcal{D}_c\}_{c=1}^m$, public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$, underlying PIR query generator PIR.Query
Output: Batch query $\mathcal{Q} = \{Q_c\}_{c=1}^m$ and client-side selection map $\{\tau_c\}_{c=1}^m$

- 1: Initialize $\mathcal{Q} \leftarrow \emptyset$
- 2: **for** $c = 1$ **to** m **do**
- 3: Perform a binary search for $r(\ell)$ in the target-rank interval projection of $\mathcal{M}_c$
- 4: **if** there exists exactly one interval containing $r(\ell)$ **then**
- 5: Let j_c be the position of that node in $\mathcal{D}_c$
- 6: Set $\tau_c \leftarrow j_c$ and $Q_c \leftarrow \text{PIR.Query}(\mathcal{D}_c, j_c)$
- 7: **else**
- 8: Set $\tau_c \leftarrow \perp$ and sample $d_c \leftarrow \{0, \dots, |\mathcal{D}_c| - 1\}$
- 9: Set $Q_c \leftarrow \text{PIR.Query}(\mathcal{D}_c, d_c)$
- 10: **end if**
- 11: Add Q_c to $\mathcal{Q}$
- 12: **end for**
- 13: Send $\mathcal{Q}$ to the server and keep $\{\tau_c\}$ locally
- 14: **return** $\mathcal{Q}, \{\tau_c\}_{c=1}^m$

5.6. Client-Side Proof Reconstruction

The online retrieval phase ends with a local reconstruction step. The server returns one PIR response for each color subdatabase. The client decodes all responses, but only those marked in the client-side selection map are used as proof records. The proof array is initialized with public default hashes, and each real response overwrites the proof level indicated by the metadata. No additional server round is required.

Figure 6 illustrates how dummy responses coexist with proof reconstruction. The client privately retrieves only the levels whose sibling subtrees contain non-default data, fills

all other slots from the public default chain, and then performs the standard bottom-up SMT verification. In the example, leaf 18 retrieves nodes 3, 5, and 8, sends a dummy query for color 1, and keeps the remaining proof slot equal to Δ_0 .

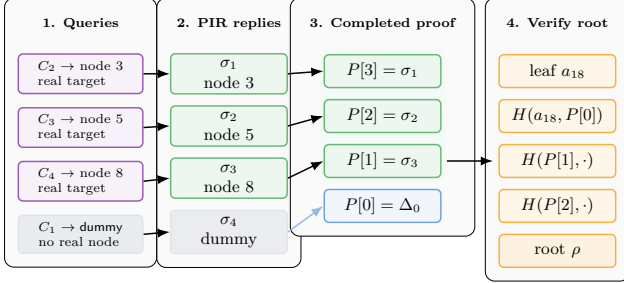


Figure 6. One target-independent batch query and client-side proof reconstruction for leaf 18.

Algorithm 4 Client-side proof reconstruction

Input: Target leaf coordinate ℓ , target leaf digest a_0 , public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$, client-side selection map $\{\tau_c\}_{c=1}^m$ from Algorithm 3, decoded PIR responses $\{\sigma_c\}_{c=1}^m$, default-empty hash chain $\{\Delta_t\}_{t=0}^{h-1}$

Output: Complete proof array $P[0..h-1]$ and reconstructed root digest ρ

```

1: for  $t = 0$  to  $h - 1$  do
2:   Set  $P[t] \leftarrow \Delta_t$ 
3: end for
4: for  $c = 1$  to  $m$  do
5:   if  $\tau_c \neq \perp$  then
6:     Read  $(I_{\text{srv}}, \delta) \leftarrow \mathcal{M}_c[\tau_c]$ 
7:     Set  $P[\delta] \leftarrow \sigma_c$ 
8:   end if
9: end for
10:  $\rho \leftarrow a_0$ 
11: for  $t = 0$  to  $h - 1$  do
12:   if the  $t$ -th path bit of  $\ell$  is 0 then
13:      $\rho \leftarrow H(\rho \| P[t])$ 
14:   else
15:      $\rho \leftarrow H(P[t] \| \rho)$ 
16:   end if
17: end for
18: return  $P, \rho$ 

```

5.7. End-to-End Protocol Flow

Putting these pieces together, the online flow for one target leaf is as follows. The client obtains its rank in the public ordered set of occupied leaves and performs one metadata lookup per color subdatabase. If the target rank lies in a stored target-rank interval, the client privately queries the corresponding record; otherwise, it sends a dummy PIR query to the same subdatabase. Thus every retrieval produces the same target-independent batch transcript: one well-formed PIR query per color. After decoding the PIR

responses, the client uses its selection map to place the real non-default sibling digests into the proof array. All remaining positions are filled with public default hashes, and the client hashes upward along the target coordinate to obtain the SMT root.

6. Correctness, Privacy, and Complexity

This section proves the correctness, privacy, and complexity properties of SparseTreePIR. The results are stated for the one-query-per-color retrieval model used by the construction: each retrievable proof node is stored in one color subdatabase, and each online retrieval sends one PIR query to every color subdatabase. For brevity in this section, we call the PIR-retrievable sibling nodes of Section 4 *active proof nodes*.

Let $N = |\mathcal{A}(T)|$ be the number of active proof nodes and let $n = |\mathcal{L}|$ be the number of occupied leaves. If $n = 1$, then $\mathcal{A}(T) = \emptyset$, $m = 0$, and no PIR query is needed. Unless otherwise stated, we assume $n \geq 2$.

6.1. Width and Record Complexity

The first result characterizes the number of colors needed by the one-query-per-color retrieval model.

Theorem 6.1 (Tightness of m in one-query-per-color retrieval). *Let*

$$m = \max_{\ell \in \mathcal{L}} |\text{Path}(\ell)|.$$

For a fixed height- h SMT snapshot, m colors are necessary and sufficient in the one-query-per-color retrieval model. A proper m -coloring can be computed in $O(N)$ time once the interval forest has been constructed. Moreover,

$$m \leq \min\{h, n - 1\},$$

and this bound is tight for every h and $n \geq 2$.

Proof. For necessity, choose an occupied leaf ℓ^* with $|\text{Path}(\ell^*)| = m$. All nodes in $\text{Path}(\ell^*)$ appear in the same membership proof. Therefore, any proper coloring must assign distinct colors to these m nodes. Hence at least m colors are necessary.

For sufficiency, consider the target-rank intervals $\{I_{\text{srv}}(u) : u \in \mathcal{A}(T)\}$. These intervals form a laminar family. Two active proof nodes can appear in the same membership proof exactly when their intervals contain the same target rank. In a laminar family, this is equivalent to one interval containing the other, i.e., to an ancestor-descendant relation in the corresponding interval forest. Thus, coloring each node by its depth in the interval forest is proper: two nodes at the same forest depth are not ancestor and descendant, so their intervals are disjoint and they cannot co-occur in one proof. The number of colors used by this depth coloring is the maximum number of target-rank intervals containing any target rank, which is exactly m . A single traversal of the interval forest computes this coloring in $O(N)$ time.

For the structural bound, every active proof node on a target path corresponds to a distinct SMT level whose sibling

subtree is non-empty. Hence there are at most h such nodes. In addition, the non-empty sibling subtrees on one target path are pairwise disjoint, and each contains at least one occupied leaf different from the target leaf. Therefore, the path contains at most $n - 1$ active proof nodes. This proves $m \leq \min\{h, n - 1\}$.

The bound is tight. Let $k = \min\{h, n - 1\}$ and choose the target leaf with logical bit string 0^h . For each depth $d = 0, \dots, k - 1$, add one occupied leaf in the sibling subtree that branches away from the target path at depth d , for example the bit string $0^d 1 0^{h-d-1}$. These k leaves make k different sibling subtrees on the target path non-empty, so the target proof contains k active proof nodes. Any remaining occupied leaves can be added elsewhere without removing these witnesses. Hence the bound is attainable.

Remark 6.1. *Theorem 6.1 is where the SMT formulation departs most clearly from a naive complete-tree view. In a complete-tree layout, the batch profile is pinned to the full tree height h regardless of how much non-default proof material exists. In the direct SMT formulation, the batch profile is governed instead by the maximum number of non-default sibling positions on any occupied-leaf proof. Consequently, the storage gain and the width gain are related but not identical: reducing the stored proof records is immediate, while reducing the number of color queries requires the proof-specific PIR record sets themselves to become smaller.*

Lemma 6.1 (Number of active proof nodes). *Let $n = |\mathcal{L}|$. Then*

$$|\mathcal{A}(T)| \leq 2(n - 1).$$

The bound is tight for every binary occupied-leaf skeleton in which every branching internal node has two non-empty children represented in the SMT.

Proof. Every active proof node u has a parent p whose two original SMT children are both non-empty: u is non-empty by definition, and $\text{sib}(u)$ is also non-empty. Conversely, each branching parent with two non-empty children contributes at most its two child roots to $\mathcal{A}(T)$. Thus $|\mathcal{A}(T)|$ is at most twice the number of branching internal nodes in the compressed non-default skeleton induced by the occupied leaves.

Any finite rooted binary tree with n leaves has at most $n - 1$ branching internal nodes. Therefore $|\mathcal{A}(T)| \leq 2(n - 1)$. When the occupied-leaf skeleton is a full binary skeleton with $n - 1$ branching internal nodes and both children of each branching node correspond to non-empty SMT subtrees, both child roots are active, giving equality.

6.2. Correctness

We next prove that the coloring and retrieval procedures return exactly the non-default sibling digests needed for a standard SMT membership proof.

Proposition 6.1 (Validity of subtree color permutations). *Let φ be a proper coloring, let U be any rooted subtree of the interval forest, and let $A(U) \subseteq [m]$ be the set of colors*

not used on the strict ancestor chain of U . Assume every node in U is colored from $A(U)$. If π is any permutation of $A(U)$, and every node $x \in U$ is recolored from $\varphi(x)$ to $\pi(\varphi(x))$ while all nodes outside U keep their colors, then the resulting coloring is still proper.

Proof. Consider any two nodes that may appear together in one proof. If both lie outside U , nothing changes. If both lie inside U , they were originally assigned distinct colors under φ , and applying the same permutation π preserves distinctness because π is injective on $A(U)$. It remains to consider a pair consisting of one node $x \in U$ and one node $y \notin U$.

If x and y can co-occur on a proof path, then y must be an ancestor of the subtree root of U , because nodes outside U but incomparable with U correspond to disjoint intervals and cannot appear on the same path. By definition, every ancestor color is excluded from $A(U)$. Therefore $\pi(\varphi(x)) \in A(U)$ can never equal $\varphi(y)$, and the pair remains properly colored. Hence no violating ancestor-descendant pair is created, so the recolored assignment is valid.

Proposition 6.2 (Per-color uniqueness on proof paths). *If φ is a proper coloring, then for every occupied leaf $\ell \in \mathcal{L}$ and every color $c \in [m]$, the set $\text{Path}(\ell)$ contains at most one active proof node of color c .*

Proof. If there were two distinct nodes $u, v \in \text{Path}(\ell)$ with $\varphi(u) = \varphi(v) = c$, the definition of proper coloring would be violated.

Proposition 6.3 (Fixed-size batch retrieval). *Assume a correct single-item PIR backend. Then any proper coloring lets the client recover all active proof nodes of a target leaf ℓ using one batch-PIR request with exactly m PIR subqueries.*

Proof. By Proposition 6.2, each color subdatabase contains at most one node from $\text{Path}(\ell)$. Therefore, for each color, the client either sends a PIR query for the unique real record in that subdatabase, or sends a dummy PIR query when no such record exists. Correctness of the PIR scheme returns the selected digest in every real color. These returned digests are exactly the non-default sibling digests in $\text{Path}(\ell)$.

Proposition 6.4 (Correctness of default-hash completion). *Assume that the underlying single-item PIR scheme is correct. If φ is a proper coloring, then Algorithm 4 reconstructs the complete SMT proof of the target leaf ℓ from the real PIR outputs and the public default-empty hash chain.*

Proof. Fix a proof level $t \in \{0, \dots, h - 1\}$. If the sibling subtree at level t is non-empty, then its digest is an active proof node. This node belongs to $\text{Path}(\ell)$, appears in exactly one color subdatabase by Proposition 6.2, and is retrieved correctly by Algorithm 3. Algorithm 4 writes this digest into the proof slot $P[t]$. If the sibling subtree at level t is empty, then its digest is the public default value Δ_t , and Algorithm 4 leaves $P[t] = \Delta_t$. Hence every proof slot contains the correct sibling digest. The client can therefore recompute the SMT root by the standard bottom-up Merkle verification procedure.

6.3. Target Privacy

We now prove target privacy relative to the setup leakage defined in Section 4.

Theorem 6.2 (Target privacy under setup leakage). *Assume the setup leakage profile and the target-privacy game in Definition 4.6. Assume also that the retrieval width m is polynomially bounded in the security parameter, as holds when the SMT height h is polynomially bounded because $m \leq h$. If each underlying single-item PIR query is query-private over its corresponding color subdatabase, and if dummy queries are generated as in Definition 4.7, then the batch retrieval protocol is target-private relative to $\mathcal{L}_{\text{setup}}(T)$.*

Proof. Fix any efficient adversary and any two occupied target leaves ℓ_0, ℓ_1 chosen in the game. Condition on the setup leakage $\mathcal{L}_{\text{setup}}(T)$. The adversary already knows the SMT snapshot information included in the leakage, the active proof nodes, the coloring, the public metadata tables, the color subdatabase sizes, and the fact that every retrieval sends one PIR query to each color subdatabase. Therefore, the only target-dependent part of the online transcript is the sequence of PIR queries.

Consider one color c . For target ℓ_b , Algorithm 3 either queries the unique real index in $\mathcal{D}_c$ whose target-rank interval contains the target rank, or sends a dummy query over the same subdatabase as in Definition 4.7. If both targets induce real queries, single-item PIR query privacy makes the two selected indices computationally indistinguishable. If one target induces a real query and the other induces a dummy query, the dummy query is still a well-formed PIR query over the same subdatabase, and PIR query privacy hides whether the selected index is meaningful. If both targets induce dummy queries, the two query distributions are identical up to target-independent randomness.

A standard hybrid argument over the m colors shows that the complete batch transcript for ℓ_0 is computationally indistinguishable from the complete batch transcript for ℓ_1 . Since m is polynomially bounded, the hybrid loss is negligible. Thus every efficient adversary has negligible advantage in the target-privacy game.

6.4. Load Balance

Correctness and privacy require only a proper coloring. The running time of a PIR backend, however, depends on the sizes of the color subdatabases. We therefore analyze the maximum subdatabase size.

Lemma 6.2 (Same-color intervals are disjoint). *If φ is a proper coloring, then for any color c , any two distinct intervals in*

$$\mathcal{I}_c = \{I_{\text{srv}}(u) : \varphi(u) = c\}$$

are disjoint.

Proof. Suppose two distinct same-color nodes u and v had overlapping target-rank intervals. Then some occupied-leaf

rank would be contained in both intervals, so the corresponding membership proof would contain both u and v . This contradicts Proposition 6.2.

Theorem 6.3 (Structural lower bound for color-subdatabase balance). *Let $N = |\mathcal{A}(T)|$ and let m be the number of colors. For a node v in the interval forest, let $d(v)$ be the number of active proof nodes on the path from the root of its connected component to v , including v , and let $\text{desc}(v)$ be the number of strict descendants of v . Define*

$$B(T) = \max \left\{ \left\lceil \frac{N}{m} \right\rceil, \max_{v: d(v) < m} \left\lceil \frac{\text{desc}(v)}{m - d(v)} \right\rceil \right\}.$$

If the second maximum is over an empty set, it is omitted. Then every proper coloring $\varphi : \mathcal{A}(T) \rightarrow [m]$ satisfies

$$\max_{c \in [m]} |\mathcal{D}_c| \geq B(T).$$

Proof. The term $\lceil N/m \rceil$ is the equal-split lower bound. For the second term, fix a node v with $d(v) < m$. The $d(v)$ nodes on the path from the component root to v must have distinct colors. Every strict descendant of v co-occurs with each of these path nodes in some proof and therefore cannot use any of their $d(v)$ colors. Hence the $\text{desc}(v)$ strict descendants must be assigned to at most $m - d(v)$ remaining colors. By the pigeonhole principle, at least one color subdatabase contains at least

$$\left\lceil \frac{\text{desc}(v)}{m - d(v)} \right\rceil$$

of these descendants. Taking the maximum over all such v and combining it with the equal-split bound proves the claim.

Proposition 6.5 (A posteriori balance certificate and termination). *Let φ_{out} be the coloring returned by Algorithm 2, and let*

$$U = \max_{c \in [m]} |\mathcal{D}_c(\varphi_{\text{out}})|.$$

Let OPT be the smallest possible value of $\max_c |\mathcal{D}_c|$ over all proper m -colorings. Then

$$\frac{U}{\text{OPT}} \leq \frac{U}{B(T)}.$$

Thus $U/B(T)$ is an a posteriori balance certificate for the returned coloring. Moreover, Algorithm 2 accepts only finitely many improving moves. If it stops because no candidate subtree permutation improves the load vector, then the returned coloring is a local optimum under the allowed subtree-permutation moves.

Proof. By Theorem 6.3, $\text{OPT} \geq B(T)$. Therefore

$$\frac{U}{\text{OPT}} \leq \frac{U}{B(T)}.$$

This proves the certificate.

For termination, every accepted move strictly decreases the lexicographically sorted load vector $\Psi(\varphi) = \text{sort}_{\downarrow}(|\mathcal{D}_1|, \dots, |\mathcal{D}_m|)$. There are finitely many colorings of

the finite interval forest over the fixed color set $[m]$. Therefore, the algorithm cannot accept infinitely many strictly improving moves. If a pass finds no improving candidate, then no subtree permutation considered by the algorithm can further decrease Ψ , which is exactly local optimality with respect to this move set.

6.5. Indexing and Runtime Complexity

The remaining results give the preprocessing, balancing, and online client costs.

Theorem 6.4 (Indexing complexity). *Let $N = |\mathcal{A}(T)|$ be the total number of active proof nodes, and let $N_c = |\mathcal{D}_c|$ be the size of color subdatabase c . Then the interval metadata construction and query indexing satisfy*

$$\begin{aligned} \text{preprocessing time} &\in O(N \log N), \\ \text{preprocessing space} &\in O(N), \\ \text{single-retrieval indexing time} &\in O(m \log N). \end{aligned}$$

Proof. Computing the target-rank interval and proof level for each active proof node takes $O(N)$ time once the compressed non-default skeleton is available. Sorting the metadata tables costs

$$O\left(\sum_{c=1}^m N_c \log N_c\right) \subseteq O(N \log N).$$

At query time, the client performs one binary search per color, which costs

$$O\left(\sum_{c=1}^m \log N_c\right) \subseteq O(m \log N).$$

The active digest records and their interval metadata require $O(N)$ space.

Theorem 6.5 (ActiveBalance complexity). *Given a proper coloring of an interval forest with N active proof nodes and m colors, one execution of Algorithm 2 with at most R_1 passes runs in*

$$O(R_1 N m^3)$$

time and uses $O(Nm)$ additional space.

Proof. In one pass, Algorithm 2 computes, for each subtree root, the colors used by its ancestors and the per-color counts inside the subtree. These histograms require $O(Nm)$ space and can be computed in $O(Nm)$ time by a bottom-up traversal.

For a fixed subtree root U , the available-color set has size at most m . Building the local cost matrix K_U takes $O(m^2)$ time, and solving the resulting assignment problem using the Hungarian algorithm takes $O(m^3)$ time. Checking the induced global load vector costs $O(m)$, which is dominated by $O(m^3)$. Evaluating all N subtree roots therefore costs $O(Nm^3)$ per pass and $O(R_1 N m^3)$ over R_1 passes.

The peak additional storage is $O(Nm)$.

Theorem 6.6 (Client-side completion complexity). *Assume that the public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$ are already available and that the default-empty hash chain $\Delta_0, \dots, \Delta_{h-1}$ has been precomputed once. Then Algorithm 4 runs in $O(h)$ time and uses $O(h)$ additional space. Consequently, the total online client work for query indexing and proof completion is*

$$O(m \log N + h).$$

Proof. Algorithm 4 initializes a proof array of length h , processes at most one real PIR output per color, and performs exactly h bottom-up hash computations. Thus its time and space costs are both $O(h)$. Combining this with the $O(m \log N)$ indexing cost from Theorem 6.4 gives the stated bound. The default-empty hash chain is public and can be precomputed once in $O(h)$ time and space.

The following theorem states the end-to-end guarantee used in the evaluation: it ties the construction to correctness, target privacy, storage, metadata, and PIR workload cost.

Theorem 6.7 (End-to-end SparseTreePIR guarantee). *Fix a public height- h SMT snapshot T with active proof-node set $\mathcal{A}(T)$, number of colors m , a proper coloring φ , color subdatabases $\mathcal{D}_1, \dots, \mathcal{D}_m$, and public metadata tables $\mathcal{M}_1, \dots, \mathcal{M}_m$. Assume that the underlying single-item PIR backend is correct and query-private for each subdatabase, and that dummy queries are generated as in Definition 4.7. Then SparseTreePIR satisfies the following end-to-end guarantee for every occupied target leaf:*

- 1) *the client reconstructs a valid standard height- h SMT membership proof;*
- 2) *the retrieval is target-private relative to the setup leakage in Definition 4.5 and the target-privacy game in Definition 4.6;*
- 3) *the PIR database contains exactly $N = |\mathcal{A}(T)|$ digest records, and the public metadata size is $O(N)$;*
- 4) *client-side lookup and proof completion together cost $O(m \log N + h)$ time;*
- 5) *the PIR-backend workload is determined by the load vector $(|\mathcal{D}_1|, \dots, |\mathcal{D}_m|)$: a sequential execution performs one PIR query over each color subdatabase, while a parallel execution is bottlenecked by the largest per-color backend call.*

Proof. The first item follows from Proposition 6.3 and Proposition 6.4: the client privately retrieves all non-default sibling digests and fills the remaining proof slots with public default hashes. The second item follows from Theorem 6.2. For the third item, the construction stores one digest for every active proof node, and each metadata entry contains a constant number of fields. The fourth item combines Theorem 6.4 and Theorem 6.6. Finally, Algorithm 3 issues one independent PIR query to each color subdatabase, so the backend workload composes over the color subdatabase sizes. Under parallel execution, the largest per-color backend call determines the bottleneck.

Remark 6.2. *Algorithms 3 and 4 are directly implementable: all inputs required by proof completion are either*

public metadata, public default hashes, or client-local state already produced during query generation. No additional server round is needed after batch-PIR extraction.

7. Experimental Evaluation

This section evaluates SparseTreePIR as a PIR-based retrieval layout for membership proofs in fixed public SMT snapshots. Direct SMT proof serving is used only as a functional reference, not as a privacy-preserving baseline, because a direct request reveals the queried coordinate and authentication path. We therefore compare SparseTreePIR with alternative privacy-preserving layouts that expose different database sizes, query patterns, and subdatabase sizes to the same PIR backends. The evaluation asks four questions: (i) what database and query profile is induced by real SMT key sets; (ii) how much improvement comes from storing only non-default proof records; (iii) how much does ActiveBalance reduce the maximum per-color subdatabase size; and (iv) whether these layout-level improvements translate into lower communication and runtime with existing PIR backends.

7.1. Experimental Setup

The main text reports the workload classes, platform, backends, repetitions, and measured metrics, while exact source identifiers, extraction scripts, command lines, and raw CSV outputs are provided in the artifact. We use six SMT key sets: five account-key snapshots derived from public Polygon zkEVM and ZKsync Era RPC data, and one FuelLabs SMT test-vector workload. Each key set is mapped into height-128 and height-256 SMT coordinate spaces, and duplicate coordinates are merged. These workloads are reproducible account-key snapshots rather than full archival state-tree dumps; their purpose is to evaluate the retrieval layouts induced by realistic account-key distributions.

The deterministic structural metrics are the number of PIR records $N = |\mathcal{A}(T)|$, the number of subqueries m , the maximum per-color subdatabase size $M(\varphi)$, the dummy-query fraction, and the public metadata size. These quantities describe the workload exposed to a PIR backend. For implementation-based measurements, we use the height-128 workloads and compare SparseTreePIR with the PBC-style SMT baseline using SimplePIR and PIANO. Each backend experiment is repeated over three target-sampling seeds with 50 occupied targets per workload.

All backend measurements were run on Ubuntu 22.04.5 with Linux 6.8, a 13th Gen Intel Core i5-13500HX CPU with 20 hardware threads, 15 GiB RAM, Python 3.10.12, Go 1.18.1, GCC/G++ 11.4, and CMake 3.22. Online communication counts backend upload and download bytes for the measured PIR calls. Setup time refers to backend pre-processing for the measured databases, query time refers to client query generation, and server-response time is reported separately when measured.

7.2. Privacy-Preserving Layouts

We compare five privacy-preserving retrieval layouts. *Complete-tree TreePIR* treats the SMT as a complete height- h perfect tree and is used only as an accounting baseline, because a height-128 or height-256 full coordinate tree is not materializable. *Height-preserving pruned TreePIR* removes public default records from the searched payloads but keeps a height- h query pattern. *PBC-style SMT* is an optimistic generic batch-placement baseline over the same active proof records: it uses the standard three-copy, $\lceil 1.5m \rceil$ -bucket resource profile, but reports deterministic balanced placement rather than sampled Cuckoo placement. Thus it is an optimistic accounting baseline for bucket sizes, not a completed sampled PBC implementation. *Unpartitioned active-record PIR* stores active proof records once but searches one global active-record database for each proof slot. *SparseTreePIR* stores the same active proof records once, partitions them into per-color subdatabases, and applies ActiveBalance to reduce the maximum subdatabase size.

Height-preserving pruned TreePIR baseline.. For each verifier proof slot $i \in [h]$, let $\mathcal{P}_i$ be the set of non-default sibling digests that can appear in slot i across occupied-leaf membership proofs. This baseline searches only these non-default records, but online retrieval still sends one query for every proof level, using a dummy query when the target proof has no non-default sibling at that level. Its query width is therefore h , and its maximum subdatabase size is $\max_i |\mathcal{P}_i|$. This baseline tests whether simply deleting default records is sufficient once the protocol has already committed to a height- h query pattern.

In Table 4, PIR rec. is the number of PIR records, Subq. m is the number of SparseTreePIR subqueries, Dummy is the fraction of dummy queries, Pruned q/max reports the query count and maximum subdatabase size of the height-preserving pruned TreePIR baseline, PBC b/max reports $\lceil 1.5m \rceil$ buckets and the largest PBC bucket, and STPIR max is the largest SparseTreePIR subdatabase after ActiveBalance. The table shows that the PIR database can be much smaller than the logical SMT proof space while preserving the standard height- h verification interface. Across the workload suite, SparseTreePIR uses only 9–17 subqueries, whereas the pruned TreePIR baseline still issues 128 or 256 queries. For the broad ZKsync workload, for example, 15478 active proof records are partitioned into 17 subdatabases, and the largest subdatabase contains 911 records; the Pruned TreePIR-128 baseline has a largest subdatabase of 3014 records. Increasing the logical height from 128 to 256 does not change these active records for the measured key sets because the occupied-leaf skeleton has already separated; the client reconstructs the remaining proof levels using public default hashes.

Table 5 separates the source of each improvement. Complete-tree TreePIR uses a layout that is natural for dense Merkle trees but inflated for sparse SMT snapshots. The height-preserving pruned baseline removes default records but still keeps a height- h query pattern. PBC-style SMT uses the active proof records but treats them as an arbitrary batch,

TABLE 4. SMT WORKLOAD PROFILES AND INDUCED COSTS FOR PRIVACY-PRESERVING RETRIEVAL LAYOUTS.

Workload	h	Keys	PIR rec.	Subq. m	Avg. rec.	Dummy	Pruned q/max	PBC b/max	Unpart. max	STPIR max
FuelLabs SMT vectors	128	100	198	9	6.88	23.56%	128 / 40	14 / 43	198	23
FuelLabs SMT vectors	256	100	198	9	6.88	23.56%	256 / 40	14 / 43	198	23
Polygon recent	128	384	766	11	8.89	19.15%	128 / 168	17 / 136	766	70
Polygon recent	256	384	766	11	8.89	19.15%	256 / 168	17 / 136	766	70
ZKsync sample	128	956	1910	13	10.19	21.62%	128 / 370	20 / 287	1910	147
ZKsync sample	256	956	1910	13	10.19	21.62%	256 / 370	20 / 287	1910	147
Polygon multi-window	128	1348	2694	14	10.73	23.36%	128 / 530	21 / 385	2694	193
Polygon multi-window	256	1348	2694	14	10.73	23.36%	256 / 530	21 / 385	2694	193
Polygon broad	128	7040	14078	17	13.11	22.86%	128 / 2762	26 / 1625	14078	829
Polygon broad	256	7040	14078	17	13.11	22.86%	256 / 2762	26 / 1625	14078	829
ZKsync broad	128	7740	15478	17	13.25	22.03%	128 / 3014	26 / 1786	15478	911
ZKsync broad	256	7740	15478	17	13.25	22.03%	256 / 3014	26 / 1786	15478	911

TABLE 5. MEASURED SOURCES OF IMPROVEMENT ON THE REAL SMT WORKLOAD SUITE. MULTIPLICATIVE GAINS ARE COMPUTED FROM ARITHMETIC-MEAN REDUCTIONS OVER THE SIX HEIGHT-128 WORKLOAD INSTANCES.

Comparison	Observed resource effect	Interpretation
vs. complete-tree TreePIR	Full $h = 128/256$ layouts are $> 4.3 \cdot 10^{34} \times / > 10^{73} \times$ larger than the largest active-record layout	Full coordinate trees are mismatched to sparse SMT proof retrieval.
vs. pruned TreePIR	$9.5 \times$ fewer subqueries; $2.5 \times$ smaller maximum subdatabase	Deleting default records still leaves a height- h query pattern.
vs. PBC-style SMT	$1.5 \times$ fewer subqueries; $1.9 \times$ smaller largest bucket	Generic batch placement ignores authentication-path structure.
vs. unpartitioned active-record PIR	$12.7 \times$ smaller maximum searched database	Storing only non-default records is insufficient without partitioning.
vs. node-local coloring	$1.2 \times$ smaller maximum subdatabase	ActiveBalance improves a valid coloring without changing m .

paying for replication and extra buckets. The unpartitioned active-record baseline stores only non-default proof records but repeatedly searches one large database. SparseTreePIR combines non-default proof-record storage, a reduced query count m , and balanced per-color subdatabases.

Overall, the evaluation isolates three effects. Storing only non-default proof records removes the full coordinate tree from the PIR database, avoiding a complete-tree layout that is more than $10^{34} \times$ larger even at height 128. SparseTreePIR then replaces the height- h pruned TreePIR query pattern with m subqueries, giving $9.5 \times$ fewer subqueries and a $2.5 \times$ smaller maximum subdatabase on the height-128 workloads. It also gives $1.5 \times$ fewer subqueries than the PBC-style baseline and a $12.7 \times$ smaller searched database than unpartitioned active-record PIR. ActiveBalance reduces the largest subdatabase by another $1.2 \times$ without changing m . These effects are orthogonal to the choice of PIR backend; the implementation-based experiments below measure whether the improved layout profile carries over to existing PIR systems.

TABLE 6. ACTIVEBALANCE ABLATION ON REAL HEIGHT-128 WORKLOADS

Workload	PIR rec.	m	Lower	Greedy max	AB max	Red.	Meta KB
FuelLabs SMT vectors	198	9	22	26	23	11.5%	4.6
Polygon recent	766	11	70	83	70	15.7%	18.0
ZKsync sample	1910	13	147	180	147	18.3%	44.8
Polygon multi-window	2694	14	193	236	193	18.2%	63.1
Polygon broad	14078	17	829	929	829	10.8%	330.0
ZKsync broad	15478	17	911	1162	911	21.6%	362.8

7.3. Effect of ActiveBalance

A valid coloring is not automatically a good backend layout. Table 6 compares ActiveBalance with a node-local greedy initializer on the height-128 workloads. The initializer assigns feasible colors using ancestor-color constraints and current global loads, but it does not apply subtree-level color permutations. Both schemes use the same active proof records and the same number of colors; the only difference is the balancing refinement. The Lower column is the structural lower bound $B(T)$ from Theorem 6.3.

In five of the six height-128 workloads, ActiveBalance reaches the reported structural lower bound exactly; in the Fuel test-vector workload it is within $1.045 \times$ of the lower bound. The reduction in maximum subdatabase size ranges from 10.8% to 21.6% relative to the greedy initializer. The metadata column uses the prototype encoding of 24 public bytes per active proof record, compared with 32 private digest bytes per record. Thus the public lookup state is linear in the number of active proof records and remains independent of the full logical coordinate space.

7.4. Experiments with Existing PIR Backends

We next evaluate whether the structural improvements translate to existing PIR implementations. The SimplePIR experiment uses the official Go API [36] for setup, query generation, response generation, and recovery over the induced subdatabases, and checks that the recovered digest records match the requested proof records. This is our main end-to-end backend experiment. The PIANO experiment uses the public PIANO artifact [37] with the same subdatabase-size vectors, and serves as a secondary implementation-based check of the layout-induced backend workload.

TABLE 7. REPEATED IMPLEMENTATION-BASED BACKEND RESULTS ON REAL HEIGHT-128 WORKLOADS. EACH ENTRY IS A BASELINE/SPARSETREEPIR COST RATIO AVERAGED OVER SIX WORKLOADS, THREE SEEDS, AND 50 REQUESTED OCCUPIED TARGETS PER SEED. SERVER-RESPONSE GAIN IS SIMPLEPIR RESPONSE-GENERATION TIME; PREPROCESSING GAIN IS BACKEND PREPROCESSING ONLY AND EXCLUDES LAYOUT CONSTRUCTION AND METADATA PUBLICATION.

Backend	Runs	Query-count gain	Max-size gain	Comm. gain	Client-query gain	Server-response gain	Preproc. gain
SimplePIR full API	3 × 50	1.5×	–	2.1×	1.7×	1.8×	2.3×
PIANO local runner	3 × 50	1.5×	1.9×	2.3×	2.4×	–	3.1×

TABLE 8. RAW SIMPLEPIR FULL-API MEANS BEHIND TABLE 7. VALUES ARE ARITHMETIC MEANS OVER THE THREE TARGET-SAMPLING SEEDS; COMMUNICATION IS KB PER REQUESTED PROOF AND TIMINGS ARE MILLISECONDS.

Workload	PBC KB	Sparse KB	PBC query	Sparse query	PBC answer	Sparse answer
Fuel vectors	12.000	5.906	9.061	5.936	0.033	0.023
Polygon recent	25.500	11.688	12.180	7.609	0.044	0.029
ZKsync sample	43.125	20.719	15.817	9.684	0.076	0.038
Polygon multi-window	51.844	25.375	17.792	10.633	0.081	0.047
Polygon broad	133.250	62.156	32.367	17.223	0.245	0.114
ZKsync broad	138.938	65.344	32.302	17.679	0.268	0.120

TABLE 9. PER-WORKLOAD IMPLEMENTATION-BASED BACKEND GAINS AGAINST THE PBC-STYLE SMT BASELINE. QUERY COUNT REPORTS PBC-STYLE BUCKETS / SPARSETREEPIR COLORS; OTHER ENTRIES ARE BASELINE/SPARSETREEPIR COST RATIOS AVERAGED OVER THREE SEEDS.

Workload	Query count	SimplePIR comm.	SimplePIR query	PIANO comm.	PIANO query	PIANO max size
FuelLabs SMT vectors	14 / 9	2.0×	1.5×	3.2×	3.4×	1.9×
Polygon recent	17 / 11	2.2×	1.6×	3.1×	2.7×	1.9×
ZKsync sample	20 / 13	2.1×	1.6×	2.1×	1.8×	2.0×
Polygon multi-window	21 / 14	2.0×	1.7×	2.1×	2.1×	2.0×
Polygon broad	26 / 17	2.1×	1.9×	2.1×	3.1×	2.0×
ZKsync broad	26 / 17	2.1×	1.8×	2.1×	2.1×	2.0×

Table 7 is the main backend result. Against the PBC-style SMT baseline, SparseTreePIR gives $2.1\times$ lower online communication under SimplePIR and $2.3\times$ lower online communication under PIANO. Under SimplePIR, server response-generation time is $1.8\times$ lower on average. PIANO answer-time measurements are not reported because the artifact’s small-database counters are often zero or quantized. Client query generation and backend preprocessing also improve, with gains ranging from $1.7\times$ to $3.1\times$. These gains are not caused by changing the PIR primitive: both schemes are measured through the same backend in each row. They come from the database layout exposed to the backend: the PBC-style baseline stores three copies over approximately $1.5m$ buckets, whereas SparseTreePIR stores one copy over m subdatabases.

Table 8 gives the absolute SimplePIR values behind the aggregate ratios. The ratios in Table 7 are computed from per-workload reductions, so they need not equal ratios of rounded arithmetic means in the raw table. The artifact also includes an aggregate raw-means CSV for both SimplePIR and PIANO, including backend setup time.

Table 9 shows that the communication reduction is stable across workloads. SimplePIR online communication is $2.0\times$ – $2.2\times$ lower, and PIANO online communication is $2.1\times$ – $3.2\times$ lower. Timing gains vary more than communication because they depend on backend parameter tiers, refresh behavior, and small-database effects; the SimplePIR

answer-time values are especially small and should be read as supporting evidence rather than as the primary performance metric. The more stable evidence is the deterministic communication reduction and the improved database profile, both of which hold across all repeated measurements.

7.5. Reproducibility and Scope

All tables in this section are generated by repository scripts from checked-in workload summaries and backend outputs. The artifact includes the exact extraction scripts, source identifiers, command lines, backend revisions, target-sampling seeds, raw per-seed measurements, and environment record. The main backend evidence is limited to SimplePIR and PIANO; other PIR systems are future integration targets.

The protocol scope is fixed-snapshot membership-proof retrieval for occupied leaves. Non-membership retrieval and dynamic SMT updates require additional metadata, leakage choices, and amortization analysis, and are left as future work.

8. Conclusion and Future Directions

This paper presented SparseTreePIR, a retrieval layout for target-private membership proofs in fixed binary Sparse Merkle Tree snapshots. SparseTreePIR keeps the standard height- h SMT proof interface unchanged: the verifier receives one sibling digest per level, while the PIR backend stores and retrieves only the non-default sibling digests that cannot be derived from the public default-hash chain. Public interval metadata identifies which proof records are relevant to a target leaf, and a target-independent batch query with real and dummy PIR queries hides the requested authentication path. The client reconstructs the standard SMT proof locally by combining retrieved digests with public default hashes.

Our analysis shows that private proof retrieval is governed by the non-default proof records in the occupied-leaf skeleton, rather than by the full logical height of the SMT. In the one-query-per-subdatabase model, the batch width is the maximum number of non-default sibling digests on any occupied-leaf proof path. SparseTreePIR also addresses load balancing across PIR subdatabases: ActiveBalance reduces the maximum subdatabase size by applying subtree-level permutations of subdatabase labels while preserving the same batch query pattern. On six SMT-derived workloads, the resulting batch width is only 9–17 for height-128 and height-256 proofs, and the layout reduces both the number of subqueries and the largest searched subdatabase relative to pruned TreePIR-style, generic batch-placement, and flat single-database baselines. SimplePIR and PIANO experiments further show lower online communication than the PBC-style SMT baseline without changing the underlying PIR schemes.

Future work should extend the retrieval model to non-membership proofs, dynamic SMT updates, and deployed sparse authenticated stores such as revocation-transparency

maps and Jellyfish Merkle Tree-style state stores [11], [17]. These settings require additional leakage choices, update and rebuilding policies, and end-to-end engineering analysis for metadata publication, record packing, preprocessing, and backend selection. A complementary theoretical direction is to strengthen the current instance-specific load-balance certificate into an a priori approximation guarantee for broader interval-based proof workloads.

References

- [1] R. C. Merkle, “A Certified Digital Signature,” in *Advances in Cryptology—CRYPTO ’89 Proceedings*, G. Brassard, Ed., LNCS 435, Springer, 1990, pp. 218–238.
- [2] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, Internet Engineering Task Force, Dec. 2021.
- [3] B. Laurie, “Certificate Transparency,” *ACM Queue*, vol. 12, no. 8, pp. 10–19, 2014.
- [4] S. A. Crosby and D. S. Wallach, “Efficient Data Structures for Tamper-Evident Logging,” in *Proc. 18th USENIX Security Symposium (USENIX Security)*, 2009.
- [5] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, “Dynamo: Amazon’s Highly Available Key-Value Store,” in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205–220.
- [6] A. Anagnostopoulos, M. T. Goodrich, and R. Tamassia, “Persistent Authenticated Dictionaries and Their Applications,” in *Information Security*, LNCS 2200, Springer, 2001, pp. 379–393.
- [7] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman, “CONIKS: Bringing Key Transparency to End Users,” in *Proc. 24th USENIX Security Symposium (USENIX Security)*, 2015, pp. 383–398.
- [8] I. Tzialla, A. Kothapalli, B. Parno, and S. Setty, “Transparency Dictionaries with Succinct Proofs of Correct Operation,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] H. Malvai, L. Kokoris-Kogias, A. Sonnino, E. Ghosh, E. Ozturk, K. Lewi, and S. F. Lawlor, “Parakeet: Practical Key Transparency for End-to-End Encrypted Messaging,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2023.
- [10] J. Len, M. Chase, E. Ghosh, K. Laine, and R. Cruz Moreno, “OPTIKS: An Optimized Key Transparency System,” in *Proc. 33rd USENIX Security Symposium (USENIX Security)*, 2024, pp. 4355–4372.
- [11] B. Laurie and E. Kasper, “Revocation Transparency,” Google Research technical report, Sep. 2012.
- [12] D. Koisser, P. Jauernig, G. Tsudik, and A.-R. Sadeghi, “V’CER: Efficient Certificate Validation in Constrained Networks,” in *Proc. 31st USENIX Security Symposium (USENIX Security)*, 2022, pp. 4491–4508.
- [13] B. Bailey and S. Sankagiri, “Merkle Trees Optimized for Stateless Clients in Bitcoin,” in *Financial Cryptography and Data Security*, LNCS 12676, Springer, 2021, pp. 451–466.
- [14] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” Ethereum Project Yellow Paper, 2014.
- [15] D. E. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, “Zcash Protocol Specification,” Version 2023.4.0, 2023.
- [16] R. Dahlberg, T. Pulls, and R. Peeters, “Efficient Sparse Merkle Trees: Caching Strategies and Secure (Non-)Membership Proofs,” *Cryptology ePrint Archive*, Paper 2016/683, 2016.
- [17] Z. Gao, Y. Hu, and Q. Wu, “Jellyfish Merkle Tree,” Diem Association technical paper, revised Jan. 12, 2021.
- [18] L. Ramabaja and A. Avdullahu, “Compact Merkle Multiproofs,” arXiv preprint arXiv:2002.07648, 2020.
- [19] B. Ma, V. N. Pathak, L. Liu, and S. Ruj, “One-Phase Batch Update on Sparse Merkle Trees for Rollups,” arXiv preprint arXiv:2310.13328, 2023.
- [20] M. I. Silva, J. Messias, and B. Livshits, “A Public Dataset For the ZKsync Rollup,” arXiv preprint arXiv:2407.18699, 2024.
- [21] M. Petkus, “Efficient (Non-)Membership Tree from Multicollision-Resistance with Applications to Zero-Knowledge Proofs,” *Cryptology ePrint Archive*, Paper 2024/1259, 2024.
- [22] Q. Cao, S. H. Dau, R. Gagiano, D. Huynh, X. Yi, P. L. Le, Q.-H. Luu, E. Viterbo, Y.-C. Huang, J. Zhu, M. M. Jalalzai, and C. Feng, “TreePIR: Efficient Private Retrieval of Merkle Proofs via Tree Colorings with Fast Indexing and Zero Storage Overhead,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2025, pp. 4476–4494.
- [23] S. Eskandarian, E. Messeri, J. Bonneau, and D. Boneh, “Certificate Transparency with Privacy,” *Proc. Privacy Enhancing Technologies*, vol. 2017, no. 4, pp. 329–344, 2017.
- [24] D. Kales, O. Omolola, and S. Ramacher, “Revisiting User Privacy for Certificate Transparency,” in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, 2019, pp. 432–447.
- [25] W. Lueks and I. Goldberg, “Sublinear Scaling for Multi-Client Private Information Retrieval,” in *Financial Cryptography and Data Security*, LNCS 8975, Springer, 2015, pp. 168–186.
- [26] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, “Private information retrieval,” *Journal of the ACM*, vol. 45, no. 6, pp. 965–981, 1998.
- [27] E. Kushilevitz and R. Ostrovsky, “Replication Is Not Needed: Single Database, Computationally-Private Information Retrieval,” in *Proc. 38th Annu. IEEE Symp. Foundations of Computer Science (FOCS)*, 1997, pp. 364–373.
- [28] C. Cachin, S. Micali, and M. Stadler, “Computationally Private Information Retrieval with Polylogarithmic Communication,” in *Advances in Cryptology—EUROCRYPT ’99*, LNCS 1592, Springer, 1999, pp. 402–414.
- [29] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, “Batch Codes and Their Applications,” in *Proc. 36th Annu. ACM Symp. Theory of Computing (STOC)*, 2004, pp. 262–271.
- [30] D. R. Stinson, R. Wei, and M. B. Paterson, “Combinatorial Batch Codes,” *Advances in Mathematics of Communications*, vol. 3, no. 1, pp. 13–27, 2009.
- [31] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [32] C. Aguilar-Melchor, J. Barrier, L. Fousse, and M.-O. Killijian, “XPIR: Private Information Retrieval for Everyone,” *Proc. Priv. Enhancing Technol.*, vol. 2016, no. 2, pp. 155–174, 2016.
- [33] S. Angel, H. Chen, K. Laine, and S. Setty, “PIR with Compressed Queries and Amortized Query Processing,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2018, pp. 962–979.
- [34] M. H. Mughees, H. Chen, and L. Ren, “OnionPIR: Response Efficient Single-Server PIR,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021, pp. 2292–2306.
- [35] S. J. Menon and D. J. Wu, “Spiral: Fast, High-Rate Single-Server PIR via FHE Composition,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2022, pp. 930–947.
- [36] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, “One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval,” in *Proc. 32nd USENIX Security Symposium (USENIX Security)*, 2023, pp. 3889–3905.
- [37] M. Zhou, A. Park, W. Zheng, and E. Shi, “Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 4296–4314.
- [38] S. J. Menon and D. J. Wu, “YPIR: High-Throughput Single-Server PIR with Silent Preprocessing,” in *Proc. 33rd USENIX Security Symposium (USENIX Security)*, 2024, pp. 5985–6002.
- [39] M. H. Mughees and L. Ren, “Vectorized Batch Private Information Retrieval,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2023, pp. 437–452.
- [40] J. Liu, J. Li, D. Wu, and K. Ren, “PIRANA: Faster Multi-query PIR via Constant-weight Codes,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 4315–4330.
- [41] A. Miller, M. Hicks, J. Katz, and E. Shi, “Authenticated Data Structures, Generically,” in *Proc. 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2014, pp. 411–423.
- [42] T. Pulls and R. Peeters, “Balloon: A Forward-Secure Append-Only Persistent Authenticated Data Structure,” in *Computer Security—ESORICS 2015*, LNCS 9326, Springer, 2015, pp. 622–641.